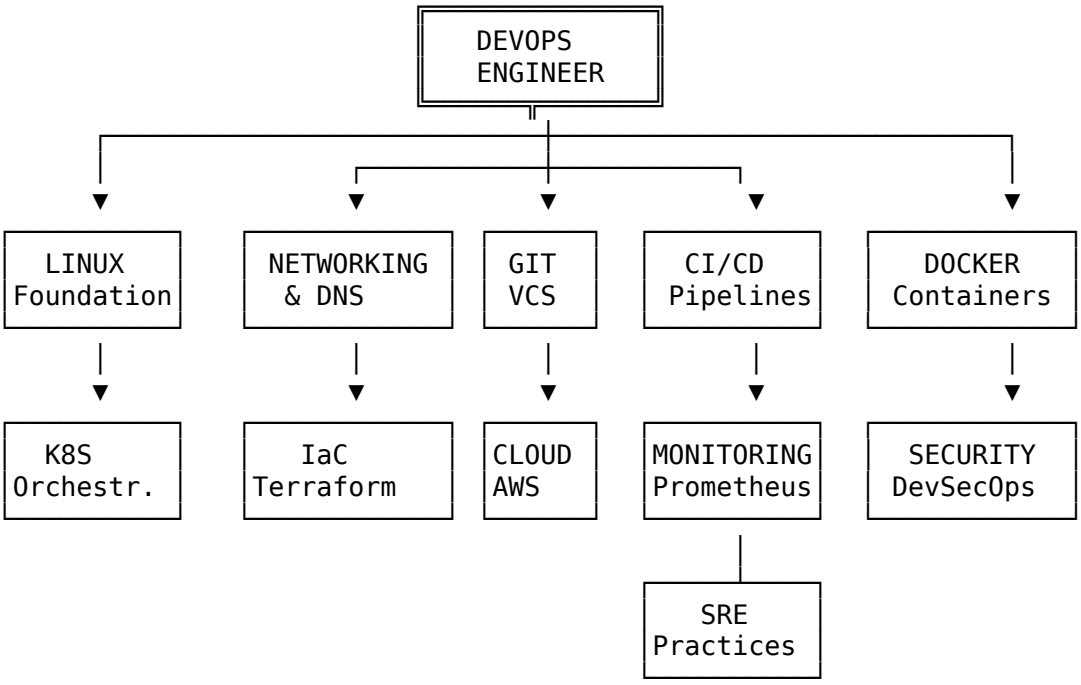
Contents

DevOps Mastery — All Domains Brain Map **1**
Master Overview . . . 1
Domain 1: Linux Fundamentals . . . 1
Domain 2: Networking . . . 2
Domain 3: Git & Version Control . . . 2
Domain 4: CI/CD . . . 3
Domain 5: Docker & Containers . . . 3
Domain 6: Kubernetes . . . 3
Domain 7: Infrastructure as Code . . . 4
Domain 8: Cloud Platforms . . . 4
Domain 9: Monitoring & Observability . . . 5
Domain 10: Security & DevSecOps . . . 5
Domain 11: SRE Practices . . . 5
Cross-Domain Concept Map . . . 6

DevOps Mastery — All Domains Brain Map

Master Overview



Domain 1: Linux Fundamentals

```
LINUX
├── Kernel — Namespaces (PID, NET, MNT, UTS, IPC, USER)
│            Cgroups (cpu, memory, blkio, pids)
│            System calls (interface between user/kernel space)
├── Filesystem — ext4/xfs, inodes, /proc, /sys, /dev
│                Permissions (rwx, SUID, SGID, sticky, ACLs)
│                LVM, mounting, fstab
├── Processes — fork/exec model, signals, zombie, daemon
│               priority (nice, renice), cgroups limits
│               strace, lsof, /proc/PID
├── Networking — iptables, ip commands, ss, tcpdump
```

```
│               DNS resolution, /etc/hosts, /etc/resolv.conf
│               Network namespaces (Docker foundation)
├── Systemd — unit files, targets, journald, boot analysis
├── Performance — vmstat, iostat, sar, perf, dmesg
└── Scripting — bash, python, awk, sed, grep, find, xargs
```

Domain 2: Networking

```
NETWORKING
├── OSI Model — 7 layers, which tools work at which layer
│             L3=IP L4=TCP/UDP L7=HTTP/gRPC
├── TCP/IP — 3-way handshake, connection states
│            TIME_WAIT, CLOSE_WAIT, SYN flood
│            TCP vs UDP — reliability vs speed tradeoffs
├── DNS — records (A, AAAA, CNAME, MX, TXT, SRV)
│       resolution chain: stub→recursive→authoritative
│       TTL, negative caching, split-horizon
├── Load Balancing — L4 vs L7
│                  Algorithms: round-robin, least-conn, IP hash
│                  Session persistence (sticky sessions)
│                  Health checks
├── HTTPS/TLS — certificate chain, handshake, cipher suites
│             Let's Encrypt, cert-manager, mutual TLS
├── HTTP — methods, status codes, headers, HTTP/2, gRPC
│        Keep-Alive, connection pooling
├── Firewalls — iptables chains (INPUT/OUTPUT/FORWARD)
│             Security Groups (AWS), Network ACLs
│             UFW (simplified iptables)
├── CDN — edge caching, cache invalidation, origin shield
└── Service Mesh — sidecar proxy (Envoy), mTLS, circuit breaking
                 Istio, Linkerd
```

Domain 3: Git & Version Control

```
GIT
├── Internals — objects: blob, tree, commit, tag
│             refs: branches, HEAD, remotes
│             .git directory structure
│             Content-addressable storage (SHA-1/SHA-256)
├── Branching Strategies
│   ├── Trunk-Based Development — short-lived branches, feature flags
│   ├── Git Flow — main+develop+feature+release+hotfix
│   └── GitHub Flow — main + feature branches + PR
├── Advanced Commands
│   ├── rebase — linear history, interactive (-i) for cleanup
│   ├── cherry-pick — apply specific commits
│   ├── bisect — binary search for bug introduction
│   ├── reflog — recover anything (90 day window)
│   ├── stash — save WIP without commit
│   └── worktree — multiple working trees
├── Collaboration — merge vs rebase, PR workflow
│                 code review best practices
│                 branch protection rules
├── Git Hooks — pre-commit, commit-msg, pre-push
│             Husky (Node.js), pre-commit framework
└── Best Practices — conventional commits, signed commits (GPG)
                   squash merge, commit message quality
```

Domain 4: CI/CD

```
CI/CD
├── Principles — fail fast, single artifact, immutable builds
│               DORA metrics (deployment frequency, lead time, MTTR, CFR)
├── CI — Lint → Test → Build → Scan → Publish
│        Every PR: run full suite
│        Coverage gates, quality gates
├── CD Strategies
│   ├── Rolling — gradual pod replacement
│   ├── Blue-Green — instant traffic switch, double infrastructure
│   ├── Canary — % traffic to new version, automated rollback
│   └── Feature Flags — decouple deploy from release
├── Tools
│   ├── GitHub Actions — YAML workflows, Actions marketplace, OIDC
│   ├── GitLab CI — .gitlab-ci.yml, runners, environments
│   ├── Jenkins — Groovy pipeline, plugins, agents
│   └── ArgoCD — GitOps, App of Apps, ApplicationSet
├── GitOps — Git as source of truth
│           Pull model (cluster pulls from Git)
│           Drift detection + auto-remediation
│           Argo Rollouts for advanced strategies
└── Best Practices — idempotent pipelines, artifact versioning
                     signed artifacts, audit trail
```

Domain 5: Docker & Containers

```
DOCKER
├── Architecture — client → daemon (dockerd) → containerd → runc
│                  Docker Desktop (Mac/Win) = Linux VM inside
├── Image Layers — each instruction = layer (COW filesystem)
│                   layer cache — order FROM → RUN/COPY matters
│                   BuildKit — parallel builds, --mount=type=cache
├── Dockerfile Best Practices
│   ├── Minimal base (distroless, alpine, slim)
│   ├── Multi-stage (separate build from runtime)
│   ├── Non-root user (USER 1001)
│   ├── HEALTHCHECK defined
│   ├── Pin versions (tag + digest)
│   └── No secrets in layers (use --secret mount)
├── Networking — bridge, host, overlay, macvlan
│                Container DNS (service discovery)
├── Volumes — bind mount vs volume vs tmpfs
├── Compose — multi-service local dev
│              networks, secrets, depends_on + healthcheck
├── Registry — Docker Hub, GHCR, ECR, GCR, Harbor (private)
│              Image signing, content trust
└── Security — no privileged, read-only FS, scan with Trivy
                seccomp profiles, AppArmor
```

Domain 6: Kubernetes

```
KUBERNETES
├── Control Plane — API Server, etcd, Scheduler, Controller Manager
```

```
├── Data Plane — kubelet, kube-proxy, CRI (containerd)
├── Workloads — Pod, Deployment, StatefulSet, DaemonSet, Job, CronJob
├── Networking — Services (ClusterIP/NodePort/LB), Ingress, NetworkPolicy
│              CNI: Calico, Cilium, Flannel
├── Storage — PV, PVC, StorageClass (dynamic provisioning)
│           CSI drivers, StatefulSet volume templates
├── Config & Secrets — ConfigMap, Secret, External Secrets Operator
├── RBAC — Role, ClusterRole, RoleBinding, ServiceAccount
├── Scheduling — Requests/Limits, QoS, Affinity, Taints, TopologySpread
├── Autoscaling — HPA, VPA, KEDA, Cluster Autoscaler
├── Helm — Chart, Values, Release, Hooks, Rollback
├── Security — PSA, SecurityContext, IRSA, Kyverno, OPA/Gatekeeper
└── Ecosystem — ArgoCD, cert-manager, Falco, Linkerd, Istio
```

Domain 7: Infrastructure as Code

```
IaC
├── Terraform
│   ├── HCL — providers, resources, data, outputs, variables
│   ├── State — local vs remote (S3+DynamoDB), state locking
│   ├── Modules — reusable components, versioned
│   ├── Workspace — environment isolation (or use directories)
│   ├── Lifecycle — create_before_destroy, prevent_destroy, ignore_changes
│   ├── Functions — for_each, count, dynamic, lookup, merge
│   ├── Testing — terraform validate, tfsec, checkov, Terratest
│   └── CI/CD — atlantis, terraform-github-actions, Terraform Cloud
├── Ansible
│   ├── Inventory — static, dynamic (EC2, K8s)
│   ├── Playbooks — tasks, handlers, roles
│   ├── Modules — command, shell, file, template, package, service
│   ├── Variables — vars, defaults, facts, vault-encrypted
│   ├── Roles — reusable task collections
│   └── Vault — encrypted secrets at rest in Git
├── Pulumi — real programming language (Python, Go, TypeScript)
│           Better for complex logic vs HCL
└── Comparison
    Terraform:  most widely used, declarative HCL, large provider ecosystem
    Ansible:    agentless, good for config management + K8s post-provision
    Pulumi:     code-first, better testing, steeper learning curve
    CloudFormation: AWS-native, JSON/YAML, deep AWS integration
```

Domain 8: Cloud Platforms

```
CLOUD (AWS Focus)
├── Compute — EC2 (families, purchasing), EKS, ECS, Lambda, Fargate
├── Networking — VPC (subnets, IGW, NAT, SG, NACL)
│              ALB/NLB/GWLB, Route53, CloudFront, PrivateLink
├── Storage — S3 (classes, lifecycle, replication, encryption)
│           EBS (gp3, io2), EFS, FSx
├── Database — RDS (Multi-AZ, read replicas), Aurora (HA, Serverless)
│            DynamoDB (single-table design, GSI, DAX)
│            ElastiCache (Redis/Memcached), Redshift
├── IAM — users, roles, policies (identity/resource-based)
│       SCP, permission boundaries, conditions
│       IRSA, OIDC federation
├── Security — GuardDuty, Security Hub, Inspector, Macie
│            KMS, Secrets Manager, WAF, Shield, CloudTrail
```

```
├── DevOps — CodePipeline, CodeBuild, ECR, EKS Anywhere
│           Systems Manager, CloudFormation
├── Monitoring — CloudWatch (metrics, logs, alarms, X-Ray)
└── Cost — Savings Plans, Reserved, Spot, Compute Optimizer
       Cost Explorer, Budgets, Anomaly Detection, tagging
```

Domain 9: Monitoring & Observability

```
OBSERVABILITY
├── Pillars — Metrics + Logs + Traces (+ Events)
├── Prometheus — pull model, TSDB, PromQL, service discovery
│   ├── Counters, Gauges, Histograms, Summaries
│   ├── Alertmanager — routing, deduplication, silencing
│   └── Thanos/Cortex/Mimir — long-term storage at scale
├── Grafana — visualization, alerting, unified dashboards
│             Variable templating, annotations, explore
├── Logging
│   ├── ELK Stack — Elasticsearch + Logstash + Kibana
│   ├── Loki — label-based indexing, cheaper, LogQL
│   └── Fluent Bit — lightweight shipper, K8s native
├── Tracing — Jaeger, Tempo, Zipkin (OpenTracing/OpenTelemetry)
│             Spans, traces, baggage, head vs tail sampling
├── OpenTelemetry — unified SDK, vendor-neutral, auto-instrumentation
│                   Collector pipeline: receivers→processors→exporters
├── SLO Alerting — SLI/SLO/SLA, error budgets, burn rate alerts
│                 Multi-window: 1h critical + 6h warning
└── Methods — USE (infra), RED (services), Four Golden Signals
```

Domain 10: Security & DevSecOps

```
SECURITY
├── Shift-Left — security at every pipeline stage
├── Pre-commit — gitleaks, detect-secrets, pre-commit framework
├── SAST — Semgrep, SonarQube, Bandit, CodeQL
├── SCA — Snyk, Dependabot, Trivy fs, OWASP DC
├── Container Security — Trivy, Grype, Distroless/Alpine
│                        Non-root, read-only FS, drop capabilities
├── IaC Security — Checkov, tfsec, kube-score, kubesec, KICS
├── DAST — OWASP ZAP, Burp Suite, Nuclei
├── Secrets Management — HashiCorp Vault (dynamic secrets, PKI)
│                        AWS Secrets Manager (auto-rotation)
│                        External Secrets Operator, CSI Driver
├── K8s Security — RBAC, PSA, NetworkPolicy, SecurityContext
│                  OPA/Gatekeeper, Kyverno, IRSA
├── Supply Chain — SBOM (Syft, SPDX, CycloneDX)
│                  Cosign image signing, Sigstore, SLSA levels
├── Runtime — Falco (eBPF syscall monitoring)
│             Anomaly detection, drift detection
├── Compliance — SOC2, PCI-DSS, HIPAA, GDPR, ISO27001, FedRAMP
└── Zero Trust — never trust/always verify, mTLS, SPIFFE/SPIRE
```

Domain 11: SRE Practices

```
SRE
├── Philosophy — engineering approach to ops, toil < 50%
```

```
│               blameless culture, error budgets as shared language
├── SLI/SLO/SLA — measure → target → contract
│                Error budget = reliability currency
├── Four Golden Signals — Latency, Traffic, Errors, Saturation
├── Toil — identify, measure, automate, eliminate root cause
├── Incident Management
│   ├── Severity levels (SEV-1 to SEV-4)
│   ├── Roles: IC, Tech Lead, Comms, Scribe
│   ├── Response workflow: Detect→Triage→Mitigate→Resolve→Postmortem
│   └── Blameless postmortem — 5 Whys → systemic action items
├── Alerting — alert on symptoms, burn rate > threshold, add runbooks
│             On-call: max 2 weeks/quarter, page only when action needed
├── Capacity Planning — measure, forecast, headroom, test, automate
├── Chaos Engineering — define steady state → inject failure → fix
│                       LitmusChaos, GameDay events, graduated rollout
└── Resilience Patterns
    ├── Circuit Breaker (CLOSED→OPEN→HALF-OPEN)
    ├── Retry with exponential backoff + jitter
    ├── Bulkhead — isolate resources per dependency
    ├── Timeout — every external call must have one
    └── Fallback — degrade gracefully, not fail hard
```

Cross-Domain Concept Map

```
LINUX namespaces+cgroups ──────────────► DOCKER isolation model
DOCKER containers ─────────────────────► KUBERNETES pods
KUBERNETES clusters ───────────────────► TERRAFORM provisioning
TERRAFORM infrastructure ──────────────► CLOUD resources (AWS/GCP/Azure)
ALL above services ────────────────────► MONITORING instrumentation
MONITORING alerts ─────────────────────► SRE incident response
SRE error budgets ─────────────────────► CI/CD deployment velocity
CI/CD pipeline ────────────────────────► SECURITY scanning gates
SECURITY policies ─────────────────────► KUBERNETES admission control
GIT as source of truth ────────────────► GITOPS (ArgoCD/Flux)
ALL pipelines ─────────────────────────► DORA metrics measurement
```